

Data Structures

Dr. Tayyaba Khushid

Department of Cyber Security,
Dawood University of Engineering and Technology (DUET).

August 31, 2026

Algorithm Analysis

Analysis of efficiency of an algorithm can be performed at two different stages, before implementation and after implementation, as

- **A priori analysis** This is defined as theoretical analysis of an algorithm. Efficiency of algorithm is measured by assuming that all other factors e.g. speed of processor, are constant and have no effect on implementation.
- **A posterior analysis** This is defined as empirical analysis of an algorithm. The chosen algorithm is implemented using programming language. Next the chosen algorithm is executed on target computer machine. In this analysis, actual statistics like running time and space needed are collected.

Data types

The Data Type is basically a type of data that can be used in different computer program. It signifies the type like integer, float etc, the space like integer will take 4-bytes, character will take 1-byte of space etc.

- Primitive data types - includes byte , short , int , long , float , double , boolean and char.
- Non-primitive data types - such as String , Arrays and Classes (you will learn more about these in a later chapter)

Abstract Data Type in Data Structures

The abstract datatype is special kind of datatype, whose behavior is defined by a set of values and set of operations. The keyword “Abstract” is used as we can use these datatypes, we can perform different operations. But how those operations are working that is totally hidden from the user. The ADT is made of with primitive datatypes, but operation logics are hidden.

Stack

Book 2 Page 110

- `isFull()`, This is used to check whether stack is full or not
- `isEmpty()`, This is used to check whether stack is empty or not
- `push(x)`, This is used to push `x` into the stack
- `pop()`, This is used to delete one element from top of the stack
- `peek()`, This is used to get the top most element of the stack
- `size()`, this function is used to get number of elements present into the stack

Queue

Book 2 Page 117

- `isFull()`, This is used to check whether queue is full or not
- `isEmpty()`, This is used to check whether queue is empty or not
- `insert(x)`, This is used to add `x` into the queue at the rear end
- `delete()`, This is used to delete one element from the front end of the queue
- `size()`, this function is used to get number of elements present into the queue

List

- `size()`, this function is used to get number of elements present into the list
- `insert(x)`, this function is used to insert one element into the list
- `remove(x)`, this function is used to remove given element from the list
- `get(i)`, this function is used to get element at position `i`
- `replace(x, y)`, this function is used to replace `x` with `y` value

code of Stack

```
9
10 class Stack:
11     def __init__(self):|
12         self.stack = [] # Internal list to store st
13
14     # Push operation: adds an element to the stack
15     def push(self, element):
16         self.stack.append(element)
17         print(f"{element} pushed to stack")
18
19     # Pop operation: removes and returns the top ele
20     def pop(self):
21         if not self.is_empty():
22             return self.stack.pop()
23         else:
24             return "Stack is empty"
25
26     # Peek operation: returns the top element without
27     def peek(self):
28         if not self.is_empty():
29             return self.stack[-1]
30         else:
31             return "Stack is empty"
```

Figure: Code of Abstract of stack

code of Stack

```

34 def is_empty(self):
35     return len(self.stack) == 0
36
37 # Display the stack
38 def display(self):
39     print("Current Stack:", self.stack)
40
41
42 # Create an instance of the Stack
43 my_stack = Stack()
44
45 # Perform operations on the stack
46 my_stack.push(10) # Pushes 10 to the stack
47 my_stack.push(20) # Pushes 20 to the stack
48 my_stack.push(30) # Pushes 30 to the stack
49
50 my_stack.display() # Displays the current stack
51
52 print("Top element is:", my_stack.peek()) # Peeks t
53
54 print("Popped element is:", my_stack.pop()) # Pops
55 my_stack.display() # Displays the current stack aft
56

```

Figure: Code of Abstract of stack

code of Queue

```

1 class Queue:
2     def __init__(self, max_size):
3         self.queue = []
4         self.max_size = max_size
5
6     # Check if the queue is full
7     def isFull(self):
8         return len(self.queue) == self.max_size
9
10    # Check if the queue is empty
11    def isEmpty(self):
12        return len(self.queue) == 0
13
14    # Add an element to the rear of the queue
15    def insert(self, x):
16        if self.isFull():
17            print("Queue is full! Cannot insert more elements.")
18        else:
19            self.queue.append(x)
20            print(f"Inserted {x} into the queue.")
21
22    # Delete an element from the front of the queue

```

Figure: Code of Abstract of queue

code of Queue

```
23 def delete(self):
24     if self.isEmpty():
25         print("Queue is empty! No elements to delete.")
26     else:
27         deleted_element = self.queue.pop(0)
28         print(f"Deleted {deleted_element} from the queue.")
29
30     # Get the number of elements in the queue
31     def size(self):
32         return len(self.queue)
33
34 # Example usage:
35 q = Queue(5) # Create a queue with a maximum size of 5
36
37 q.insert(10)
38 q.insert(20)
39 q.insert(30)
40
41 print("Queue size:", q.size())
42
43 q.delete()
44 print("Queue size after deletion:", q.size())
45
```

Figure: Code of Abstract of queue

code of Queue

```
0
1 print("Queue size:", q.size())
2
3 q.delete()
4 print("Queue size after deletion:", q.size())
5
6 q.insert(40)
7 q.insert(50)
8 q.insert(60)
9 q.insert(70) # This will show that the queue is full
10
11 print("Queue size after inserting more elements:", q.size())
12
13 q.delete()
14 print("Queue size after deletion:", q.size())
15
```

Figure: Code of Abstract of queue

code of Queue

```
In [3]: runfile('C:/Users/AL-MALAK/Desktop/DS_class/
untitled0.py', wdir='C:/Users/AL-MALAK/Desktop/DS
class')
Inserted 10 into the queue.
Inserted 20 into the queue.
Inserted 30 into the queue.
Queue size: 3
Deleted 10 from the queue.
Queue size after deletion: 2
Inserted 40 into the queue.
Inserted 50 into the queue.
Inserted 60 into the queue.
Queue is full! Cannot insert more elements.
Queue size after inserting more elements: 5
Deleted 20 from the queue.
Queue size after deletion: 4

In [4]:
```

Figure: Output of Abstract of queue

code of Queue

```
File "C:/Users/AL-MALAK/Desktop/DS
class/untitled0.py", line 8, in
<module>
    a=float([2,3,4,6,7,1])
```

TypeError: float() argument must be a string or a number, not 'list'

In [2]:

```
In [2]: runfile('C:/Users/AL-MALAK/
Desktop/DS class/untitled0.py',
wdir='C:/Users/AL-MALAK/Desktop/DS
class')
```

10 pushed to stack

20 pushed to stack

30 pushed to stack

Current Stack: [10, 20, 30]

Top element is: 30

Popped element is: 30

Current Stack: [10, 20]

In [3]:

Figure: Output of Abstract of Queue